RELAY AEGIS
Resilience Report

Evidence-driven self-healing · Manas Suryavanshi, BCA Final Year, Dezyne E'cole College · Hackathon 2026 (Sentry × Décole) · Chaos measurements of 23 Oct 2026

RELAY AEGIS detects failures, explains them from evidence, repairs what it can repair safely, and **proves the result before it calls anything fixed**. Every number below comes from a chaos script that anyone can run against the protected application (`chaos/chaos.ps1`): each failure was injected once with healing **off** and once with healing **on** (automatic, no human action), and timed both by the script and by RELAY's own incident records.

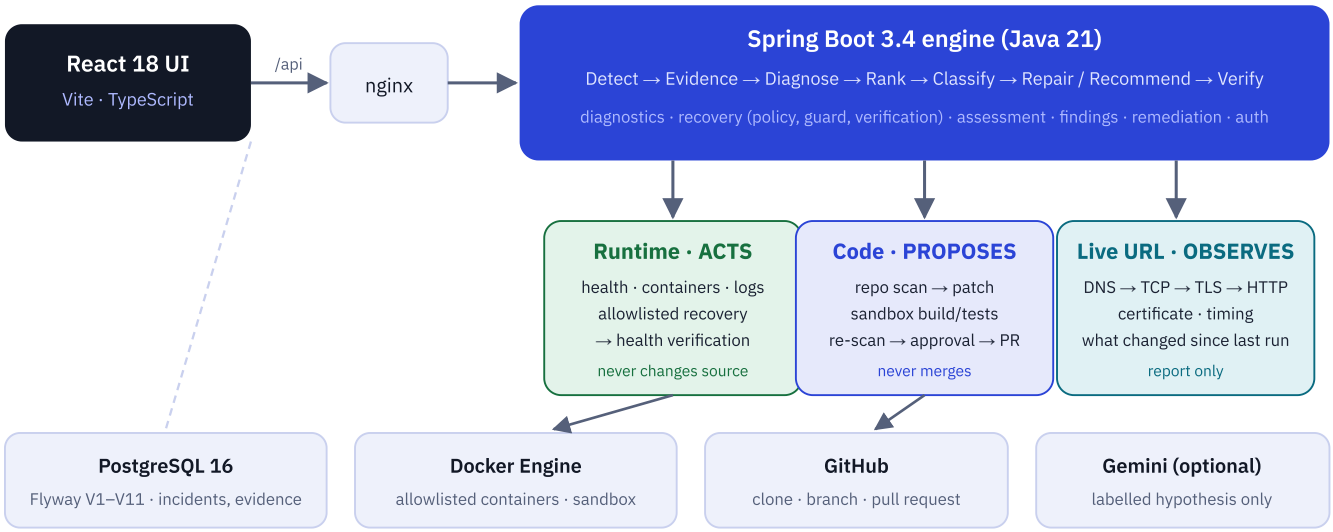
5 of 5
infrastructure failures healed automatically, no human action

5–11 s
time to detect, every failure (health check every 5 s)

28–46 s
outage with healing, against *still down after 90 s* without it in 4 of 5

3
failures it cannot heal yet, shown and explained (corrupt data, rotated password, flood)

Architecture



One engine, three evidence boundaries. Each boundary decides what RELAY may observe and what it may change; they are separate, not ranked. **Runtime → Code:** when a dependency rejects an application's credentials, Runtime refuses to restart (a restart reuses the same credential) and links the incident to the hard-coded credential Code found in the repository; the incident closes only after the PR is merged and the system is healthy again.

Healing pipeline and safety gates

Every runtime action passes, in order: **recovery policy** (closed action set `START_CONTAINER`, `RESTART_CONTAINER`, `RESTART_DEPENDENCY`, `RECONNECT_NETWORK`; Production targets blocked) → **guard** (2 attempts per incident, 60 s per-container cooldown, circuit breaker opens after 3 failed recoveries in 30 min) → **operator approval** (unless auto-repair is enabled with `CONFIRM-AUTO-REPAIR`) → **execution** → **verification** (consecutive healthy checks within a 30 s window). An action that ran is not a recovery until verification passes.

Failure table

Protected application: Smart Business AI Assistant (`sba-frontend` → `sba-backend` → `sba-postgres`), plus the `relay-aegis-orders-service` demo for the credential case. Each failure is one command of `chaos.ps1`. **Detect** and **Recover** are RELAY's incident records; **Outage** is measured independently by the script, polling the backend's own `/actuator/health` every second. Runs with healing on: policies set to *Automatic*, nobody touched anything.

Failure (chaos command)	How it is detected	How it heals	Measured (healing on)	Outcome
Runtime `stop-backend` process stopped	2 failed health checks; container *exited*; diagnosis `container-stopped`	`START_CONTAINER` sba-backend, then 2 consecutive healthy checks	detect **5.3 s** recover 30.4 s outage **33.1 s**	**Healed**
Runtime `crash-backend` process exits	Health checks fail while the container is down	`START_CONTAINER` sba-backend (Docker's own restart policy also restarts it)	detect **5.2 s** recover 25.1 s outage **29.4 s**	**Healed**
Runtime `freeze-backend` process hung (SIGSTOP)	Container running, TCP open, HTTP never answers; diagnosis `application-timeout`	`RESTART_CONTAINER` sba-backend	detect **11.2 s** recover 33.8 s outage **46.1 s**	**Healed**
Runtime `stop-db` dependency down	Backend health DOWN; `sba-postgres` exited; diagnosis `dependency-unavailable`, ranked above "restart the backend"	`RESTART_DEPENDENCY` sba-postgres; backend left alone	detect **11.2 s** recover 18.4 s outage **27.7 s**	**Healed**
Runtime `block-db` service blocked (network)	Both containers healthy, backend cannot reach the database; diagnosis `dependency-network-detached`	`RECONNECT_NETWORK` sba-postgres with its alias `postgres`; nothing restarted	detect **11.1 s** recover 15.2 s outage **30.6 s**	**Healed**
Runtime `corrupt-record` unreadable order row	**Not detected**: health stayed green for the whole run	None. No restart can repair bad data.	no incident	**Cannot heal yet**
Runtime `flood` 20 000 requests, 300 at a time	Health stayed green (median 3.2 s, same as idle)	None needed; RELAY has no rate-limit or scaling action	no outage	Absorbed by the app
Runtime → Code `rotate-password`	Incident in **8.1 s**; log `password authentication failed`; rule `dependency-credentials-rejected`, 0.98	**No restart** (it would reuse the stale password). Linked to the hard-coded credential Code found; fixed by sandbox-validated patch → approval → PR → human merge	still down after 90 s at runtime, by design	**Not at runtime: routed to Code**
Code Hard-coded credential	Static rule on a read-only clone	Patch in a disposable sandbox → build and tests pass → re-scan 1 → 0 → approval → one-line PR	sandbox: minutes (Maven)	**PR, human merges**
Live URL Bad TLS certificate	Own TLS handshake: host-name match and chain trust reported separately	Report only; fault localised to the TLS layer	n/a	**Fault localised**

Before and after: without healing vs with healing

Same failure, same command, run twice. **Without healing**, RELAY's recovery policies were set to *Requires approval* and nobody approved; RELAY still detected and diagnosed every failure (in 5.1–11.5 s) but took no action.

Failure	Without healing	With healing (automatic)	RELAY action
`stop-backend`	**still down after 90 s**	**back after 33.1 s**	`START_CONTAINER`
`crash-backend`	back after 30.5 s (Docker restart policy)	**back after 29.4 s**	`START_CONTAINER`
`freeze-backend`	**still down after 90 s**	**back after 46.1 s**	`RESTART_CONTAINER`
`stop-db`	**still down after 90 s**	**back after 27.7 s**	`RESTART_DEPENDENCY`
`block-db`	**still down after 90 s**	**back after 30.6 s**	`RECONNECT_NETWORK`
`corrupt-record`, `flood`, `rotate-password`	Identical with and without healing: RELAY takes no runtime action on these (see the failure table)		none

The 30-second target

- Met for `stop-db` (27.7 s) and `crash-backend` (29.4 s); close for `block-db` (30.6 s) and `stop-backend` (33.1 s); missed for `freeze-backend` (46.1 s).
- RELAY's own part is short: detection 5–11 s, then one action. Most of the remaining time is the protected app's Spring Boot start-up (about 20–25 s) after a start or restart; a freeze also has to time out before it counts as down.

How it is logged

- Every incident stores its timeline with timestamps: **Detected → Diagnosed → Action taken → Recovered** (verified), plus `detection_duration_ms` and `recovery_duration_ms`.
- The dashboard shows each system green / amber / red, the incident log and the count of verified recoveries.
- The chaos script writes every run to `chaos/results/chaos-results.csv`.

Engineering quality

Check	Result
Backend tests (`mvn -B clean test`)	**423 tests, 0 failures, 0 errors**
Frontend tests (vitest)	**25 files, 170 tests passed**
TypeScript, ESLint, production build, `docker compose config`	**All passed; 0 lint errors or warnings**
Scenario tests for the SBA runtime failures and credential rejection	**Pass; new signals do not fire on unrelated logs**

> **Rule behind every number:** nothing is marked resolved because an action ran. A runtime recovery is resolved only after health verification passes; a code repair ends as a reviewed pull request that a human merges, and it is resolved only after a re-scan of the default branch shows the finding gone.

Known limits

Scope

- Local, single-host deployment. The Compose stack mounts the Docker socket and is not hardened for untrusted networks.
- Runtime recovery is a closed set of four actions on allowlisted containers; anything else is *Investigate* or

What it cannot heal yet

- **Corrupted data.** A bad record inside a healthy app is invisible to health checks, containers, logs and networks, and no restart repairs it. Needs application-level checks (synthetic requests) and a data-repair path.

Escalate. Production targets are blocked by policy.

- Only two deterministic code-repair strategies (`credential-literal-to-env`, `dockerfile-non-root-user`); the Dockerfile strategy is covered by automated tests but not yet exercised on a live repository.
- Live URL observes one endpoint (plus the origin root) from outside; it cannot see internal health beyond what the endpoint returns.

- **Wrong credentials.** Deliberately not healed at runtime; the fix is code, through a reviewed PR.
- **Traffic floods.** No rate-limiting or scaling action; restarting would not stop the traffic.

Measurement

- Each failure was run once with and once without healing; single runs, not averages.
- Detection scales with the check interval (5 s in these runs) and the failure threshold (2).
- The protected app's health endpoint takes 3–4 s per call (Spring checks its SMTP server), which adds to each check.

Analysis and security

- Repository rules are heuristic static scanning; findings are candidates until validated. Known gap: the credential rule does not flag every naming pattern (for example a `*_SECRET_KEY` constant).
- Removing a hard-coded credential does not revoke it; rotation stays with the credential's issuer.
- The optional AI adds a labelled hypothesis only. It never executes, authorises or resolves anything, and without a key the deterministic engine works unchanged.

Next steps

- Synthetic application checks (for example a real `/api/orders` request) so corrupted data becomes detectable; repeat every chaos run several times for median and worst case.
- Multi-host support (remote Docker or Kubernetes) with the same policy, guard and verification model.
- More deterministic repair strategies, and a shared, read-only dependency cache to shorten sandbox runs.

RELAY AEGIS · github.com/krieger-geist/relay-aegis · Sources: `chaos/RESULTS.md` and `chaos/results/chaos-results.csv` (runs of 23 Oct 2026) and RELAY's incident records; reproduce with `chaos/chaos.ps1`.